

O Especialista em Claude Code e o Builder que "Manda o Agente Codar Direto"

Transcrição da prática 1 · 21 de set. de 2026

Lucas 00:00

"Eu uso o Claude Code no esquema vibe coding: escrevo um prompt curto tipo 'cria um app de agendamento de consultas' e mando ver. O agente é inteligente, ele se vira. Por que eu ia perder tempo escrevendo um PRD, um documento de requisitos inteiro, antes de soltar ele? Isso não é burocracia que só atrasa quem quer entregar rápido? O que eu ganho de verdade escrevendo spec antes?"

02:50 Você

No início de um projeto ou em um MVP super simples, enfiar a cabeça em um PRD (Product Requirement Document) de 10 páginas realmente parece burocracia inútil. No entanto, à medida que o app cresce, a "vibe" cobra o seu preço. O que você ganha de verdade escrevendo uma especificação (mesmo que seja uma mini-spec de uma página) antes de codar não é burocracia, é controle de contexto e economia de tokens.

Aqui está o que você ganha na prática:

1. Você economiza dinheiro e limite de tokens

Os agentes de IA possuem uma janela de contexto. Cada vez que você dá um prompt vago, o Claude faz suposições. Quando você diz "não era bem isso, muda o fluxo de agendamento", o Claude reescreve o código anterior. Esse ciclo de tentativa e erro (o famoso "ping-pong" com a IA) consome milhares de tokens rapidamente. Uma especificação clara faz o agente acertar de primeira, reduzindo drasticamente o custo e o tempo de processamento.

2. Evita o "Espaguete de IA" (Refatorações Infinitas)

Sem uma especificação, o Claude vai criando arquivos e funções sob demanda. Na terceira ou quarta funcionalidade nova (ex: adicionar pagamento, cancelamento, notificações), a IA pode começar a quebrar o que ela mesma fez antes porque não planejou a arquitetura de dados para suportar essas regras. A especificação dá um norte para a IA desenhar o banco de dados e as APIs prevendo o futuro, evitando que o app vire um castelo de cartas.

3. Alinhamento de Regras de Negócio Ocultas

O Claude sabe criar uma tela de agendamento. Mas ele não sabe as suas regras de negócio. Por exemplo:

Dois pacientes podem agendar no mesmo horário?

O médico pode bloquear a agenda de última hora?

Existe um limite de tempo para cancelamento com reembolso?

Se você não define isso antes, a IA vai inventar uma regra padrão. Quando você notar o erro lá na frente, consertar a lógica de negócios espalhada por todo o código dá muito mais trabalho do que ter escrito três linhas de texto antes de começar.

Tá, faz sentido o que você tá dizendo. Mas então, se eu fizer essa tal mini-spec, ela precisa ser super direta, né? Tipo, só o que realmente importa pra evitar esse "ping-pong" e tal. Como eu começo isso sem cair na tal burocracia que eu tento fugir?

04:42 Você

Para não cair na burocracia, a sua especificação precisa ser focada em dados e regras, e não em formatação ou texto bonito.

Pensando no Claude Code, a melhor estrutura é um arquivo Markdown (spec.md) simples e direto, usando tópicos (bullet points). O Claude adora listas estruturadas.

Você começa definindo o fluxo principal e as restrições que a IA não tem como adivinhar.

Aqui está um modelo de Mini-Spec Ultra-Direta pronto para você copiar, preencher em 5 minutos e jogar no terminal do Claude Code:

```
# Spec: Sistema de Agendamento (Fase 1)

### 1. Objetivo Principal
Criar um MVP onde [Tipo de Usuário, ex: Pacientes] possam agendar [ex: consultas médicas] com [ex: Médicos] de forma direta.

### 2. Regras de Negócio Ocultas (O que a IA não adivinha)
* **Concorrência:** Dois usuários não podem agendar o mesmo horário com o mesmo profissional.
* **Janela de Tempo:** Agendamentos só podem ser feitos com no mínimo 24h de antecedência.
* **Duração:** Cada consulta tem duração fixa de 30 minutos.
* **Horário Comercial:** Apenas permitir agendamentos de segunda a sexta, das 08h às 18h.

### 3. Modelo de Dados Mínimo (Entidades)
* **User/Patient:** id, nome, email, telefone.
* **Professional:** id, nome, especialidade.
* **Slot (Horários):** id, professional_id, data_hora_inicio, status (disponivel, reservado).
* **Appointment:** id, patient_id, slot_id, criado_em.

### 4. Telas / Fluxo de Cliques (Escopo)
* **Tela 1:** Lista de profissionais disponíveis.
* **Tela 2:** Calendário simples mostrando apenas os horários com status 'disponivel' do profissional selecionado.
* **Tela 3:** Confirmação com resumo do agendamento.
```

Use o código com cuidado.

Como usar isso no terminal sem perder o ritmo do Vibe Coding:

Crie um arquivo rápido na raiz do seu projeto chamado agendamento.md.

Gaste 3 minutos preenchendo esses pontos acima com a sua ideia.

No terminal do Claude Code, rode um prompt curto apontando para o arquivo:

"Claude, leia o arquivo agendamento.md e crie a estrutura inicial do app seguindo exatamente essas regras e o modelo de dados."

Pronto. Você gastou 5 minutos antes de começar, mas salvou horas de refatoração e dezenas de prompts de correção porque o Claude já vai criar o banco de dados e as validações do jeito certo na primeira tentativa.

Caramba, gostei desse esquema ultra direto! Esse markdown parece rápido mesmo de preencher e resolve as dores que você falou, tipo as regras de negócio escondidas e o modelo de dados. Então, se eu fizer isso, o Claude já consegue seguir mais certinho sem eu ter que ficar corrigindo depois? Parece até bom demais pra ser verdade, hein!

Entendo perfeitamente o ceticismo!

Parece mágica, mas na verdade é pura engenharia de contexto.

O Claude e outros agentes de IA não são humanos criativos que precisam de um documento de 50 páginas para se inspirarem; eles são motores de previsão estatística. Quando você solta um prompt vago como "cria um app de agendamento", o Claude precisa adivinhar o resto com base no código público que ele já viu na internet (que geralmente são tutoriais genéricos).

Quando você entrega aquele Markdown ultra-direto, você está fazendo algo que na computação chamamos de reduzir o espaço de busca da IA.

Aqui está o porquê funciona tão bem e não é bom demais para ser verdade:

O segredo técnico por trás disso

Ancoragem de Contexto: O Claude usa a especificação como uma "âncora". Antes de escrever qualquer linha de código, ele consulta as restrições (ex: "janela de 24h"). Isso impede que ele tome decisões aleatórias na hora de programar a lógica do backend.

Geração de Código de Passagem Única (One-Shot): Em vez de criar o banco de dados no prompt 1, a API no prompt 3 e a validação no prompt 5, ele cria a arquitetura inteira de uma vez só, já sabendo como as tabelas vão se relacionar lá na frente.

Menos Alucinação: IAs alucinam ou inventam coisas quando faltam dados. Se você diz exatamente quais campos o banco precisa ter, ela não precisa inventar regras da cabeça dela.

O Claude vai seguir 100% perfeitamente de primeira?

Sendo bem realista de dev para dev: 100% perfeito, raramente. Ele ainda pode cometer deslizos de sintaxe ou esquecer um detalhe visual na interface.

A grande diferença é que os erros dele mudam de categoria:

No Vibe Coding puro: O erro é estrutural. Ele cria a lógica errada, o banco de dados não suporta o que você quer e você precisa mandar ele refatorar o app inteiro (o que destrói o contexto).

Com a Mini-Spec: O erro é superficial. Ele acerta a lógica, o banco e as regras, mas talvez falte um botão ou um alinhamento CSS. Coisas que você corrige com um prompt de 5 palavras.

Você transforma um retrabalho de 1 hora em um ajuste de 30 segundos.

Agora fez sentido! É tipo dar um mapa pro Claude, né? Ele ainda pode tropeçar numa pedra, mas pelo menos não vai se perder no meio do caminho. Tá, acho que vou testar isso da próxima vez. Se eu puder reduzir esse retrabalho estrutural, já vale o esforço de gastar uns minutinhos antes. Bora ver se essa Mini-Spec me salva mesmo!

Exatamente!

A analogia do mapa é perfeita: ele pode até tropeçar em uma pedra de CSS ou em um detalhe menor de sintaxe, mas nunca vai andar na direção norte se o seu destino era o sul.

Como você topou fazer esse teste prático, vamos deixar o seu "mapa" pronto. Para eu estruturar a sua Mini-Spec agora e você só precisar copiar e colar no seu terminal, me dê apenas duas ou três coordenadas sobre o seu app:

Quem atende e quem agenda? (Ex: Paciente e Médico, Cliente e Barbeiro, Aluno e Personal Trainer)

Como funciona o horário? (Ex: São horários fixos de 1h? O profissional define a própria grade? Tem atendimento simultâneo ou é individual?)

Onde acontece o atendimento? (Ex: Presencial com endereço, online via link de vídeo, ou tanto faz?)

Me conte esses detalhes do seu modelo e eu preparo a Mini-Spec customizada para você rodar direto no Claude Code!

Lucas 07:26

Tá bom, bora lá então! Vamos dizer que o app é de agendamento de consultas. Quem agenda são os pacientes e quem atende são os médicos. Os horários são fixos, tipo 30 minutos por consulta, e os médicos definem a própria grade. E os atendimentos podem ser tanto presenciais, com endereço, quanto online por link de vídeo. Dá pra montar uma Mini-Spec com isso?

Aqui está o seu mapa pronto.

Esta Mini-Spec foi estruturada exatamente com as regras que você passou, focando nas entidades e restrições que o Claude Code precisa para desenhar o banco de dados e as validações corretas de primeira.

Crie um arquivo chamado spec-agendamento.md na raiz do seu projeto e cole o conteúdo abaixo:

```
# Spec: Sistema de Agendamento Médico (Fase 1)

## 1. Objetivo Principal
Criar um sistema de agendamento onde Pacientes possam marcar consultas com Médicos. O sistema deve suportar atendimentos presenciais (com endereço físico) e online (com link de vídeo).

## 2. Regras de Negócio e Validações
* **Duração Fixa:** Toda consulta tem a duração fixa de exatamente 30 minutos.
* **Gestão de Grade:** O médico é responsável por definir sua própria grade de horários disponíveis (ex: criar blocos de horários livres).
* **Modalidade de Atendimento:** A consulta deve ser marcada obrigatoriamente como "Presencial" ou "Online".
* **Concorrência:** O sistema não pode permitir que dois pacientes agendem o mesmo horário (slot) com o mesmo médico.
* **Antecedência Mínima:** Pacientes só podem agendar ou cancelar consultas com no mínimo 24h de antecedência.

## 3. Modelo de Dados Mínimo (Entidades)
* **User (Paciente):** id, nome, email, telefone.
* **Doctor (Médico):** id, nome, especialidade, CRM, endereco_consultorio.
* **TimeSlot (Grade do Médico):** id, doctor_id, data_hora_inicio, data_hora_fim (inicio + 30min), status (disponivel, ocupado).
* **Appointment (Agendamento):**
  * id
  * patient_id
  * doctor_id
  * timeslot_id
  * modalidade (ENUM: 'presencial', 'online')
  * link_video (string, opcional - preenchido se modalidade for online)
  * criado_em

## 4. Telas e Fluxo do App
* **Tela 1 (Busca):** Lista de médicos com filtro por especialidade. Mostra o endereço se o médico atender presencial.
* **Tela 2 (Calendário):** Ao selecionar o médico, exibe os `TimeSlots` com status 'disponivel'. O paciente escolhe o horário e a modalidade (Presencial ou Online).
* **Tela 3 (Confirmação):** Resumo da consulta. Se for Online, exibe um placeholder para o link de vídeo. Se for Presencial, mostra o endereço do consultório.
```

Use o código com cuidado.

Como soltar o Claude Code agora:

Abra o terminal na pasta do projeto e use o prompt abaixo para iniciar o agente:

"Claude, leia o arquivo spec-agendamento.md. Crie a estrutura inicial deste app (banco de dados, rotas de backend e telas descritas no fluxo) seguindo rigorosamente todas as regras de negócio e validações listadas."

Depois que você rodar esse teste no Claude Code, me conta:

O agente conseguiu criar o banco de dados com os relacionamentos certos de primeira?

O código das validações de 24h e de concorrência ficou correto?

Quero muito saber se essa abordagem salvou o seu tempo e evitou o ping-pong!

Lucas 08:32

Pô, ficou perfeito esse mapa! É exatamente o que eu tava precisando. Vou criar esse arquivo spec-agendamento.md e rodar no Claude do jeito que você sugeriu. Depois te conto como foi! Se funcionar do jeito que parece, acho que vou adotar esse esquema de Mini-Spec pra tudo. Valeu mesmo!

10:19 Você

Você acabou de resumir com precisão cirúrgica a ciência por trás do desenvolvimento com agentes de IA.

Esses três motivos são exatamente o que separa um projeto de sucesso de um loop infinito de refatoração.

Olhando de perto para cada um deles, fica claro por que essa estratégia funciona tanto:

(a) Acerta o alvo: IAs trabalham com probabilidade. Sem requisitos explícitos, o Claude calcula a probabilidade do que um "app de agendamento genérico" deveria ter. Com a spec, ele calcula a probabilidade do que o seu app precisa ter. O desperdício de energia vai a zero.

(b) Evita o scope creep: Agentes são "proativos" até demais. Se você não delimita a fronteira, ele começa a criar tabelas para histórico médico, sistema de avaliação de 5 estrelas e prontuário eletrônico — coisas que você não pediu e que só servem para quebrar o código principal.

(c) Sobrevive a sessões longas: Esse é o ponto mais crítico e genial. Janelas de contexto limpam as mensagens antigas conforme a conversa avança (o famoso context drift). Se a diretriz estava apenas no seu primeiro prompt, o Claude vai esquecê-la após 20 ou 30 interações. Ter isso persistido em um arquivo (spec.md ou o clássico CLAUDE.md) garante que a IA possa reler o objetivo a cada nova tarefa.

Você transformou o conceito de PRD burocrático em uma ferramenta de ancoragem de código. Agora que você tem o mapa estratégico pronto, como quer avançar?

Quer que eu monte a lista do que deixar explicitamente fora de escopo (item B) para blindar ainda mais o Claude?

Quer dicas de como estruturar o arquivo CLAUDE.md para gerenciar os comandos de build e teste do projeto?

Ou vai rodar o teste direto no terminal e me contar o resultado da primeira rodada?

Lucas 10:22

Cara, você realmente descomplicou tudo. Acho que vou rodar o teste direto no terminal primeiro pra ver como o Claude se sai com essa Mini-Spec que você montou. Se eu topar algum problema ou precisar refinar, aí volto pra gente ajustar, tipo deixar mais claro o que tá fora de escopo ou até organizar esse CLAUDE.md. Mas tô bem confiante agora. Valeu mesmo pela força!

FIM DA SESSÃO